

Digital Signal Processing (DSP) Project Report



Supervisor: Dr. Abdullah Gad

Institute: October High Institute for Engineering & Technology

Group Members:

- Ziad Mohamed Fathy – 12009
 - Moaz Atef Gouda – 12421
 - Ibrahim Mohamed Saad – 6850
 - Mohamad Ali Rushdi – 6859
 - Mohamed Abdel-Fadil – 12233
-

Abstract

This project presents the design and implementation of a desktop-based application that integrates **Image Processing** and **Speech Processing** using core **Digital Signal Processing (DSP)** techniques.

The application provides a modern graphical user interface (GUI) that allows users to apply various DSP filters, visualize signals, and export detailed reports.

The system demonstrates practical DSP concepts such as noise reduction, filtering, edge detection, waveform visualization, and spectrogram analysis in an intuitive and user-friendly environment.

1. Introduction

Digital Signal Processing (DSP) plays a vital role in modern engineering applications such as image enhancement, speech recognition, medical imaging, and audio communication systems.

The goal of this project is to bridge the gap between theoretical DSP concepts and real-world applications by developing an interactive software tool that allows users to experiment with DSP techniques on both images and audio signals.

2. Project Objectives

The main objectives of this project are:

- Implement practical DSP techniques for image and speech signals.
 - Design an intuitive and modern graphical user interface.
 - Visualize signals in both time and frequency domains.
 - Allow users to control filter parameters interactively.
 - Generate a professional PDF report documenting results.
 - Deploy the application as a standalone executable (.exe).
-

3. System Overview

The developed system follows a modular structure consisting of input handling, signal processing, visualization, and output generation.

System Workflow

1. Load Image or WAV Audio file.
2. Convert and preprocess the signal.
3. Apply selected DSP techniques.
4. Visualize results.
5. Export outputs and PDF report.

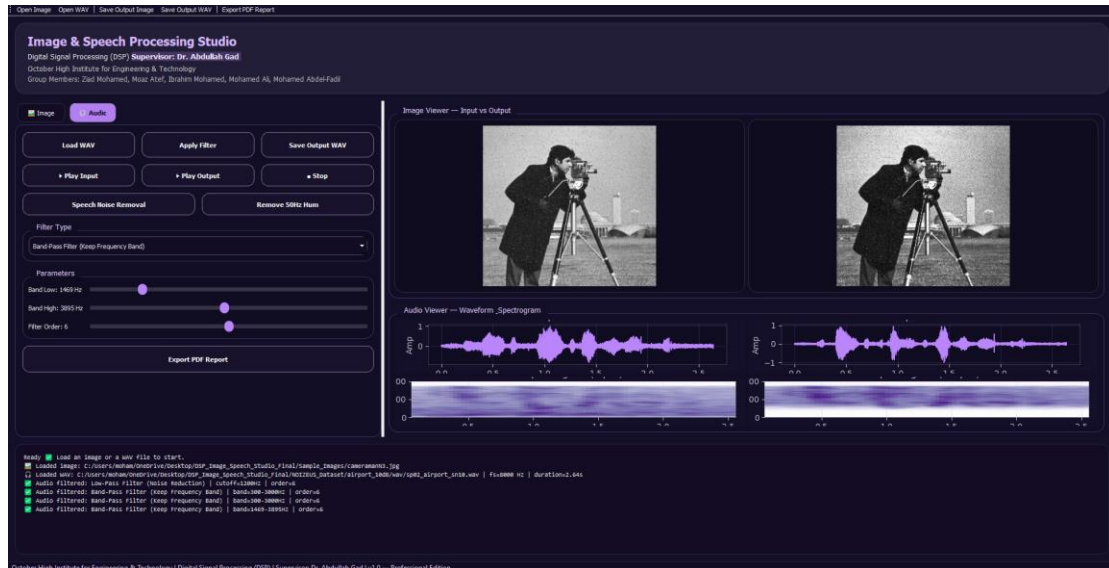


Figure 1: Overall system architecture diagram showing input, processing, visualization, and output stages.

4. DSP Techniques Implemented

4.1 Image Processing Techniques

4.1.1 Grayscale Conversion

Color images are converted to grayscale to simplify processing and reduce computational complexity.

Grayscale conversion focuses on intensity values rather than color information, which is essential for many DSP algorithms.



Figure 2: Original color image versus grayscale image.

4.1.2 Image Noise Reduction

Noise reduction improves image quality by suppressing unwanted variations in pixel intensity.

Implemented Filters:

- **Gaussian Filter:** Smooths noise using a weighted average, effective for reducing Gaussian noise.
- **Median Filter:** Removes impulse noise while preserving edges.



Figure 3: Image noise reduction comparison (Original, Gaussian Filter, Median Filter).

4.1.3 Edge Detection

Edge detection highlights important structural features in images.

- **Canny Edge Detection** was used due to its accuracy and robustness.

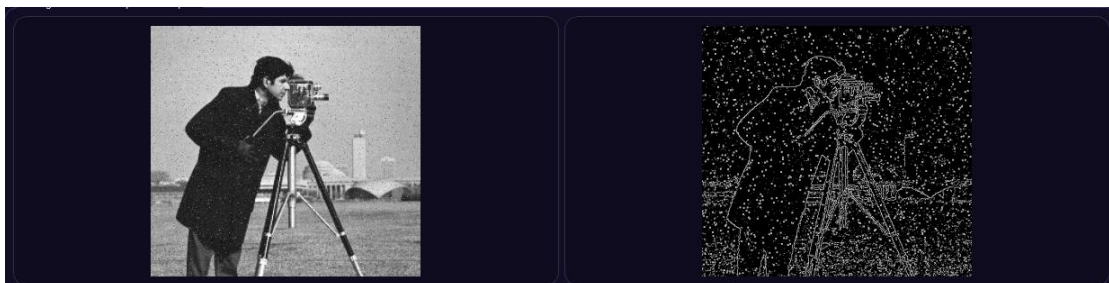


Figure 4: Edge detection results using the Canny algorithm.

4.2 Speech and Audio Processing Techniques

4.2.1 Audio Signal Representation

Audio signals are analyzed using:

- **Waveform (Time Domain)**
- **Spectrogram (Frequency Domain)**

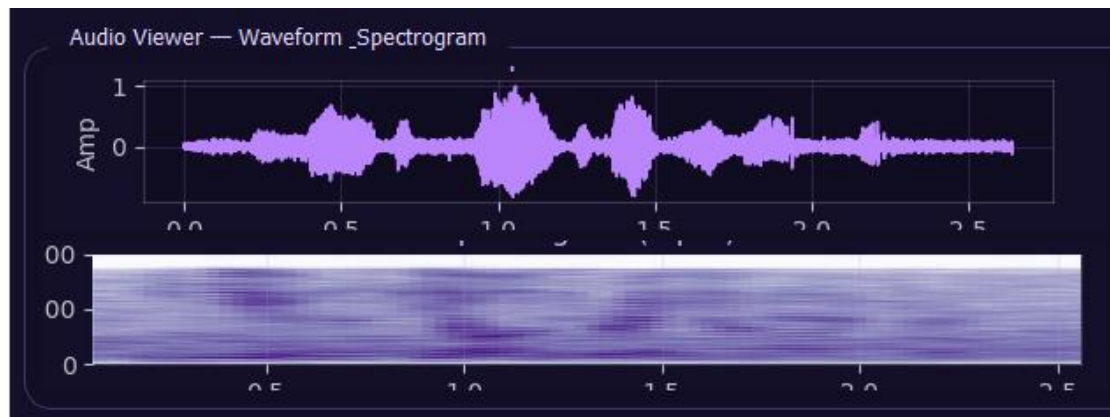


Figure 5: Original audio waveform and spectrogram.

4.2.2 Audio Filtering

The system implements several audio filters:

- **Low-Pass Filter:** Removes high-frequency noise.
- **High-Pass Filter:** Removes low-frequency hum.
- **50Hz Hum Removal:** Eliminates electrical interference.

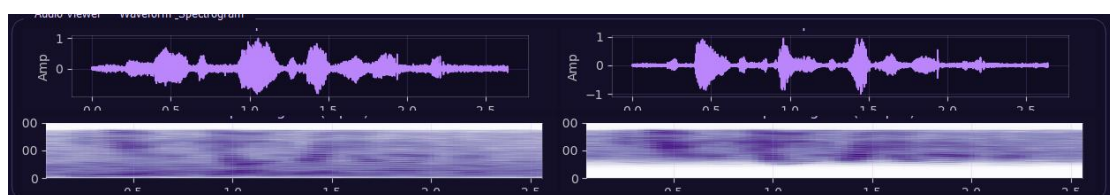


Figure 6: Audio signal before and after filtering (waveform and spectrogram).

5. Graphical User Interface (GUI) Design

The application features a modern, purple-themed GUI designed for simplicity and clarity.

Key GUI Features:

- Image and Audio mode switching.
- Interactive sliders for filter parameters.
- Live update of processing results.
- Play input/output audio controls.
- Export processed outputs and reports.

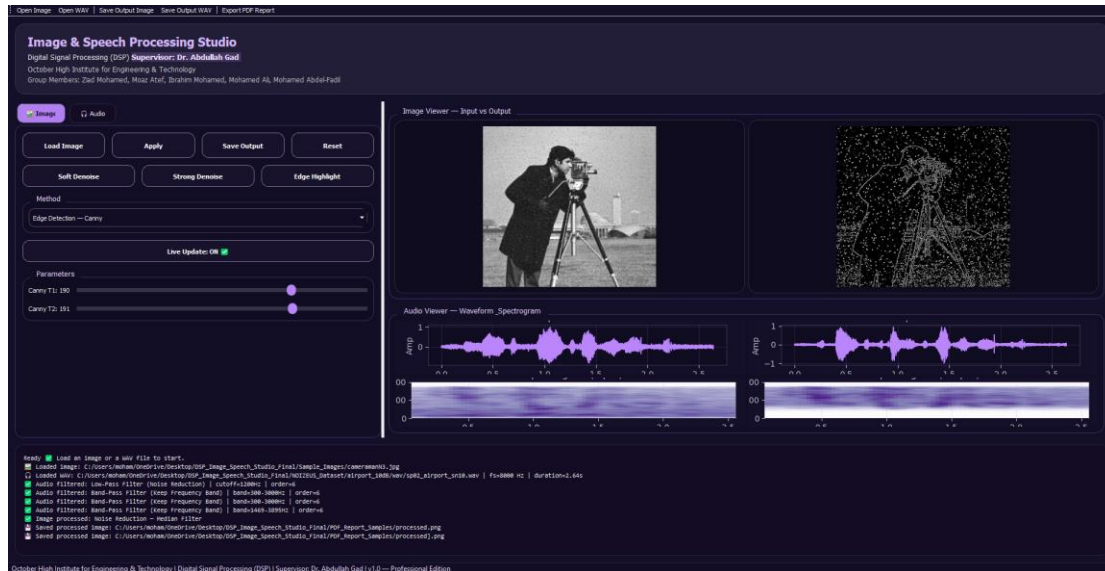


Figure 7: Main graphical user interface of the application.



Figure 8: Image processing mode interface.



Figure 9: Audio processing mode interface.

6. Visualization and Analysis

Visualization tools help users understand DSP effects clearly.

- Side-by-side comparison (Input vs Output).
- Real-time waveform updates.
- Frequency-domain analysis using spectrograms.

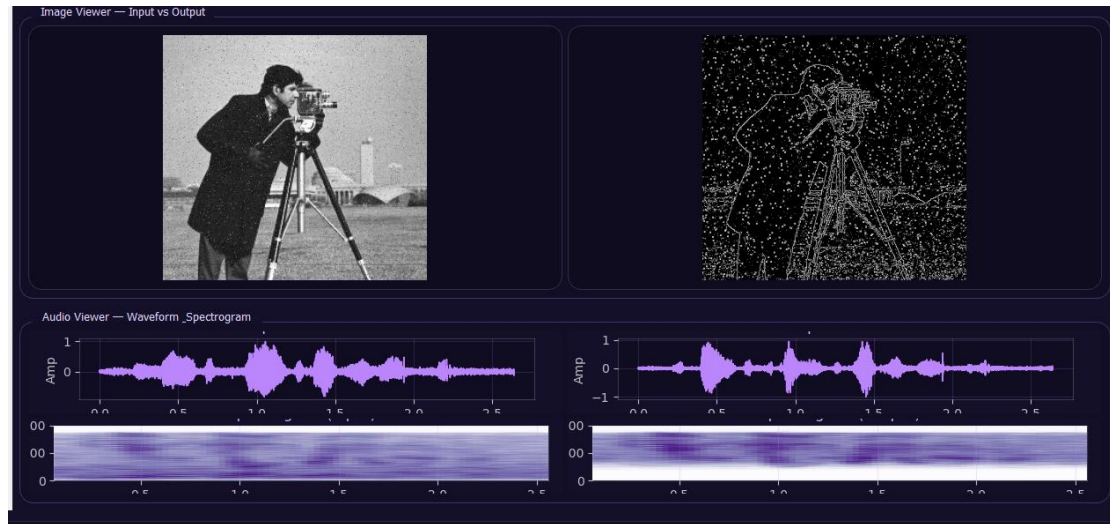


Figure 10: Visualization of input and output signals.

7. Application Deployment

The application is deployed as a **standalone Windows executable**.

Deployment Details

- Platform: Windows (64-bit)
- Format: .exe
- No Python installation required.
- Distributed via Google Drive and QR Code.



Figure 11: Application access via QR code and executable file.

8. Challenges and Solutions

Challenges

- Handling large image and audio files.
- Maintaining GUI responsiveness.
- Managing real-time signal visualization.
- Packaging Python application into a standalone executable.

Solutions

- Optimized signal processing functions.
- Efficient threading for GUI updates.
- PyInstaller used for deployment.
- Modular code structure for maintainability.

9. Performance Analysis and Observations

The system successfully enhances image quality and improves audio clarity. Noise reduction filters significantly improve signal-to-noise ratio, while visualization tools help analyze results effectively.

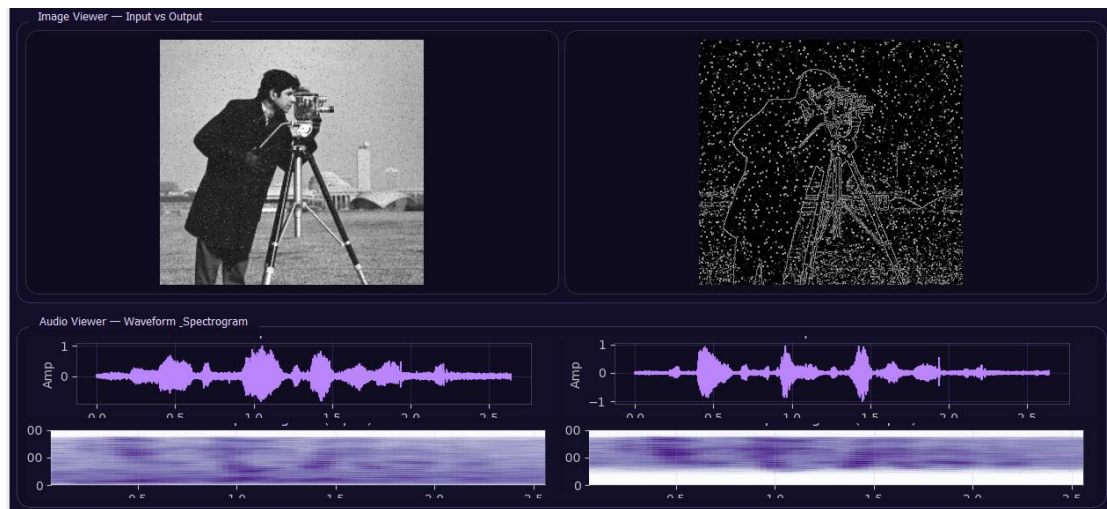


Figure 12: Performance comparison before and after DSP processing.

10. Conclusion

This project demonstrates the practical application of **Digital Signal Processing** concepts through a fully functional desktop application.

By integrating image and speech processing within a user-friendly GUI, the system provides an effective educational and practical DSP tool.

The project successfully meets all objectives and serves as a strong foundation for future extensions such as real-time processing and machine learning integration.

References

1. A. V. Oppenheim, *Signals and Systems*.
 2. Gonzalez & Woods, *Digital Image Processing*.
 3. Python Documentation – NumPy, SciPy, OpenCV, Matplotlib.
-